

SafeDrop

Software Test Document · Iterations 1–2

Team

Team Member	Role(s)	Signature	Date
Alexa Stein	Team Lead	<u>Alexa Stein</u>	<u>10/5/2027</u>
Amber Rastella	Security Lead	<u>Amber Rastella</u>	<u>10/5/2026</u>
Alex Picard	Configuration Lead	<u>Alex Picard</u>	<u>10/5/2026</u>
Mateus Silva	Requirements Lead	<u>Mateus Silva</u>	<u>10/5/2026</u>
Orelmis Toribio	QA Lead	<u>Orelmis Toribio</u>	<u>10/4/2026</u>

Revision History

Version	Author	Date	Change
0.1	Amber (draft)	9/21/2026	Initial Iteration 1 test document. Automated testing report and metrics compiled from the CI run on main at commit e4a0efd. The manual testing report is not yet filled in - see the note in that section.
0.2	Orelmis Toribio (QA Lead)	9/21/2026	Added code-structure metrics (average LOC per file, LOC per function, internal fan-out per file) to Testing Metrics. Added manual tests, and styling.
0.3	Orelmis Toribio (QA Lead)	10/4/2026	Added Iteration 2: testing summary,

			manual test cases MT-19 to MT-37 (results to be recorded by the QA Lead when run), automated testing changes, and test and code-structure metrics from the CI run on main at commit 5c4bcc8. Sections 1, 2 and 4 now have separate Iteration 1 and Iteration 2 subsections, with an Iteration 1 vs Iteration 2 comparison in 4.3. Iteration 1 content is unchanged.
--	--	--	--

Contents

1. Testing Summary
2. Manual Testing Report
3. Automated Testing Report
4. Testing Metrics
5. AI Usage Log
6. References
7. Glossary

1. Testing Summary

1.1 Iteration 1

Iteration 1 was tested almost entirely through automated tests, run on every push and pull request by GitHub Actions and required to pass before a branch may merge to main. The QA Lead owns the suite as a whole; each developer writes the tests for the feature they build, and the reviewer checks test coverage as part of the code review described in the SPPP. The figures

throughout this document come from the CI run on main at commit e4a0efd (21 September 2026), which passed all six jobs.

921 test cases passed	100% pass rate (executed cases)	94.6% / 96.0% line coverage, backend / frontend	0 high or critical vulnerabilities
---------------------------------	---	---	--

What was tested, by category:

Category	What was tested
Unit testing	Services, middleware, policies and utilities, with their dependencies mocked. 20 of the 32 backend test files are unit tests; the largest is the checkout state machine (72 cases covering every legal and illegal transition). The frontend unit tests render components with React Testing Library against a mocked API layer.
Integration testing	The Express API exercised end to end through Supertest against a real MongoDB replica set started in memory for the run, so transactions, unique constraints and TTL indexes behave as they do in production rather than being mocked. 12 backend test files are integration tests.
System testing	The full stack (SPA, API and a MongoDB replica set) is brought up with Docker Compose and exercised by hand against seeded data. CI additionally builds both Docker images on every push, so a change that breaks either image fails before it merges.
Acceptance testing	Acceptance criteria for work not yet implemented are staged in the suite as bodyless test cases (see code/docs/staged-acceptance-tests.md), so a story's definition of done lives somewhere that is run and reviewed rather than in prose that can drift. They are replaced by real tests when the ticket is implemented.
Regression testing	The whole suite runs on every push and pull request, so every previously passing test is a regression test. Two security regressions found during Iteration 1 were each closed with a test that fails without the fix (SCRUM-114 and the tenant-isolation cases below).

1.2 Iteration 2

Iteration 2 (from the Iteration1 release tag on 21 September to commit 5c4bcc8 on 4 October

2026) was tested the same way: the whole suite runs on every push and pull request, and a branch merges only with CI green. The figures below come from the CI run on main at commit 5c4bcc8 (4 October 2026), which passed every job.

1,508 test cases passed	100% pass rate (executed cases)	95.2% / 97.8% line coverage, backend / frontend	0 high or critical vulnerabilities
-----------------------------------	---	---	--

What Iteration 2 added, and where it is tested:

Story	What was built, and how it is tested
Unit maintenance (SCRUM-141)	An Org Admin moves an AVAILABLE unit into MAINTENANCE and back; any other unit state is refused with 409, and the dashboard counts units in maintenance. New: assetMaintenance.test.js (18 cases), including cross-tenant 404s and race safety.
Overdue loans (SCRUM-140, SCRUM-143)	Late CHECKED_OUT loans are moved to OVERDUE by an admin-only endpoint, and borrowers see "Due in N days" / "Overdue by N days" badges. New: overdue.test.js (23), markOverdue.test.js (11), and the frontend DueBadge and dueStatus tests.
Auto-approve (SCRUM-148)	An Org Admin decides whether requests need an approver, organization-wide with a per-asset override; an auto-approved request goes straight to APPROVED with its unit HELD. New: approvalSettings.test.js (30), plus new cases in approvalPolicy.test.js.
User groups and restricted equipment (SCRUM-149, SCRUM-150)	Org Admins create groups and restrict assets to them; an ineligible member sees a Restricted badge and is refused on submit and on approval. New: groups.test.js (41), group.test.js (13), allowedGroupIds.test.js (5), plus new cases in the assets and requests tests.
Custody confirmation (SCRUM-205)	The borrower can record their own pickup; a return waits in RETURN_PENDING until someone other than the borrower confirms or rejects it; approvals nobody collects expire after a configurable pickup window. New: custodyConfirmation.test.js (51).
Catalogue search (SCRUM-103, -145, -146, -200,	Plain and AI-assisted (Microsoft Foundry) search, retired assets excluded and results capped; any model failure falls back to plain search. New: assetSearch unit (19) and route (11) tests, plus CatalogPage tests. The model is mocked in every automated case.

-201)	
AI-recommended alternatives (SCRUM-151)	"Similar items available now" on an asset with nothing available, applying the same eligibility filters as the AI path, with a non-AI fallback. 29 cases (22 backend, 7 frontend), itemised in <code>code/docs/std-scrum-151-test-cases.md</code> .
Other changes	A deactivated member is refused at sign-in with the same 401 as a wrong password (<code>deactivate.test.js</code>). Iteration 1 review feedback split the Members page into <code>InviteForm</code> and <code>ResetPasswordForm</code> , and the UI rework restyled every page. PR #64 fixed approval-queue rows showing "Unknown user" and "Unknown asset", with 9 regression cases in <code>requests.test.js</code> .

2. Manual Testing Report

2.1 Approach

Manual testing during Iterations 1 and 2 was exploratory: features were exercised by hand against a locally seeded stack as they were built, and results were not recorded as structured test cases at the time. The QA lead reperformed the following manual tests after the fact as a final check

2.2 Test environment

The full stack runs locally with Docker Compose. From the repository root, run `docker compose up --build`; copying `.env.example` to `.env` is optional, since every variable has a development default. This starts four services:

Service	Role in manual testing
mongo	A single-node MongoDB replica set. A replica set is required because the backend wraps every state-changing action in a transaction, so each state change and its audit event commit together or not at all.
mongo-init	A one-shot sidecar that initiates the replica set once Mongo is healthy, then exits.
backend	Runs migrations, then the API at <code>http://localhost:4000</code> . <code>/health</code> returns 200 when the API and database are up and 503 when the database is unreachable.
frontend	The Vite dev server at <code>http://localhost:5173</code> , with <code>/api/*</code> proxied to the

	backend so the browser talks to a single origin.
--	--

Email. Locally nothing is emailed: MAIL_PROVIDER defaults to console, so password-reset emails (subject, body and reset link) are printed to the backend log. Password-reset tests read the link from `docker compose logs -f backend`.

Seeding. With the stack running, run `npm run seed` from code/backend. This calls `seedTwoOrgs()` (tests/helpers/seedTwoOrgs.js), the same fixture the integration suite uses, plus demo-only chain-of-custody history. The seed is destructive: it drops and recreates the two demo organizations on every run without prompting, and it refuses to run under `NODE_ENV=production`. Cases that change state should be followed by a re-seed before the next case.

2.3 Seed dataset

The seed creates two identically built organizations, Org A (org-a) and Org B (org-b), so every check against one has an equivalent to compare against the other. Each organization contains the following.

Users (all share the password `Correct-Horse-Battery-9`):

Role	Org A	Org B	Can do
MEMBER	member@a.test	member@b.test	Browse the catalogue, submit and cancel their own requests, see only their own requests.
APPROVER	approver@a.test	approver@b.test	Everything a member can, plus approve and deny requests and record checkout and return handoffs.
ORG_ADMIN	admin@a.test	admin@b.test	Everything an approver can, plus create, edit and retire assets, add units, manage member roles, read the audit log and read the dashboard.

Assets and units (for each organization):

Asset	Units	Unit statuses
Laptop	3	REQUESTED, OUT, HELD
Camera	3	AVAILABLE, AVAILABLE, RETIRED

Projector	2	AVAILABLE, OUT
------------------	---	----------------

Checkout requests (all raised by the organization's member):

Request status	Unit
PENDING	The laptop's REQUESTED unit
APPROVED	The laptop's HELD unit
CHECKED_OUT	The laptop's OUT unit (due 2026-09-24 or 2026-09-26)
CHECKED_OUT	The projector's OUT unit (due 2026-09-24 or 2026-09-26)
LOST	The camera's RETIRED unit; the request moved PENDING → APPROVED → CHECKED_OUT → LOST

Audit trail. One ORG_CREATED event from the fixture, plus nine chain-of-custody events that the seed script appends to the laptop, so its history screen shows a populated, coherent timeline.

2.4 Iteration 1 Manual test cases

ID	Test case	Type	Priority	Result
MT-01	Sign in and sign out for every role	System	High	Pass
MT-02	Create a new organization	Acceptance	High	Pass
MT-03	Forgot password, reset and change password	Acceptance	Medium	Pass
MT-04	Verify role based access to actions (admin, approver, member)	System (security)	High	Pass
MT-05	Verify Tenant isolation between organizations (assets, requests etc.)	System (security)	High	Pass
MT-06	Browse and filter the asset catalogue	Acceptance	Medium	Pass
MT-07	Create and edit an asset, add a unit	Acceptance	High	Pass
MT-08	Retire an asset (with no active units)	Acceptance	High	Pass
MT-09	Retire an asset (blocked by active units)	Acceptance	High	Pass

MT-10	Submit a checkout request; duplicate refused	Acceptance	High	Pass
MT-11	Approve and deny a request	Acceptance	High	Pass
MT-12	Record checkout and return handoffs	Acceptance	High	Pass
MT-13	Cancel a pending and an approved request	Acceptance	Medium	Pass
MT-14	View My Requests (member view)	Acceptance	Medium	Pass
MT-15	View Admin dashboard summary	Acceptance	Medium	Pass
MT-16	View Asset History	Acceptance	Medium	Pass
MT-17	Audit log filter, search and pagination	Acceptance	Medium	Pass
MT-18	Change a member's role	Acceptance	Medium	Pass

2.5 Iteration 2 Manual test cases

The cases below cover the features merged in Iteration 2. They use the same environment and seed as Sections 2.2 and 2.3 (the seed script did not change in Iteration 2); any extra setup is stated in the case. Re-seed before each case that changes state.

Dates. The seed's dates are fixed, so from 4 October 2026 both CHECKED_OUT loans are past due and the APPROVED laptop is past its default 48-hour pickup window. MT-21, MT-22 and MT-31 rely on this.

2.5.1 Test case summary

ID	Test case	Type	Priority	Result
MT-19	Put a unit into and out of maintenance	Acceptance	High	Partial - There seems to be a delay with the "Available" counter on the dashboard after

				maintenance modification
MT-20	Maintenance guarded: wrong unit state, stale page, non-admin roles	System (security)	Medium	Pass
MT-21	Due-date badges on borrowed items	Acceptance	Medium	Partial - Difference in timezone between request date input, what's stored, what's displayed
MT-22	Mark late checkouts overdue	Acceptance	High	Pass
MT-23	Approval queue: Overdue filter and readable rows (PR #64)	Regression	High	Pass
MT-24	Organization-wide auto-approve	Acceptance	High	Pass
MT-25	Per-asset approval override	Acceptance	High	Pass
MT-26	Create, rename and delete a user group; add and remove members	Acceptance	High	Pass
MT-27	Restricted equipment: members outside and inside the group	System (security)	High	Pass
MT-28	Borrower records their own pickup	Acceptance	High	Pass
MT-29	Return confirmed or rejected by someone other than the borrower	Acceptance	High	Pass
MT-30	An approver cannot confirm their own return	System (security)	High	Pass
MT-31	Pickup window and expiring uncollected approvals	Acceptance	Medium	Pass
MT-32	Catalogue search (plain search, retired assets excluded)	Acceptance	Medium	Pass
MT-33	Similar items available now	Acceptance	Medium	Pass
MT-34	Tenant isolation on the Iteration 2 pages (groups, alternatives, search)	System (security)	High	Pass

MT-35	Members page: invite a member and reset a password	Regression	Medium	Pass
-------	--	------------	--------	------

2.5.2 Test steps

ID	Steps	Expected result
MT-19	1. Sign in as admin@a.test and open Dashboard; note the In maintenance and Available figures. 2. Open Catalog, then Camera. 3. In the Units table, click Start maintenance on one AVAILABLE unit. 4. Go back to Dashboard. 5. Return to Camera and click End maintenance on the same unit. 6. Open Audit log and find the newest events for the camera.	Step 3: a notice says the unit is now in maintenance and cannot be requested; its status shows MAINTENANCE and it no longer offers Request this. Step 4: In maintenance is up by 1 and Available down by 1. Step 5: the unit is AVAILABLE again and can be requested. Step 6: one event for entering maintenance and one for leaving it, each naming the admin.
MT-20	1. As admin@a.test, open Laptop and look at the Maintenance column. 2. Open Camera in two browser tabs. In tab 1 click Start maintenance on an AVAILABLE unit; then, without refreshing, click Start maintenance on the same unit in tab 2. 3. End maintenance on that unit, then sign in as approver@a.test and as member@a.test and open Camera.	Step 1: units that are REQUESTED, OUT or HELD show a dash, with no maintenance button. Step 2: tab 2 shows an error that the unit is not available (409); after a refresh the unit is in maintenance once, with a single audit event. Step 3: there is no Maintenance column for an approver or a member.
MT-21	1. Sign in as member@a.test and open My requests. 2. Open the CHECKED_OUT laptop request and the CHECKED_OUT projector request. 3. Request an AVAILABLE camera unit from today to tomorrow; approve it as approver@a.test, then record the handoff. 4. As member@a.test, open My requests again.	Steps 1-2: both seeded loans (due 24 and 26 September) show "Overdue by N days", with N matching today's date; requests that are not out (pending, approved, lost) show no due badge. Step 4: the new loan shows "Due tomorrow".

MT-22	1. Sign in as admin@a.test. Open Dashboard and note the Overdue figure, then open Approvals and choose the Overdue filter. 2. In the browser console run: <code>await fetch('/api/requests/mark-overdue', { method: 'POST', credentials: 'include' }).then(r => r.json())</code> 3. Run the same command again. 4. Refresh Approvals and open both overdue requests. 5. Sign in as approver@a.test and run the command from step 2.	Step 1: Overdue is 2 and both loans (laptop and projector) are listed under Overdue, even though their stored state is still Checked out. Step 2: the response is <code>{"updated": 2}</code>. Step 3: the response is <code>{"updated": 0}</code> - running it twice changes nothing. Step 4: both requests now show the state OVERDUE; the LOST camera request is unchanged. Step 5: the request is refused with 403.
MT-23	1. Sign in as approver@a.test and open Approvals. 2. Click through each state filter: All, Pending, Approved, Checked out, Overdue, Return pending, Returned, Denied, Cancelled, Expired, Lost.	Step 2: every row names its requester, asset and unit - no row shows "Unknown user" or "Unknown asset" - and each filter lists only requests in that state (Overdue lists late loans whatever their stored state).
MT-24	1. Sign in as admin@a.test, open Settings, choose Approve automatically under Checkout approval and save. 2. Sign in as member@a.test and request an AVAILABLE camera unit. 3. Open the new request, then Camera's unit list. 4. As admin@a.test, open Audit log. 5. Set Settings back to Require an approver and submit another request as member@a.test.	Step 3: the request is APPROVED straight away with the note "Approved automatically by your organization's rules", and the unit is HELD. It still needs a pickup to be recorded. Step 4: REQUEST_SUBMITTED followed by REQUEST_AUTO_APPROVED naming the policy, plus an ORG_SETTINGS_UPDATED event for the settings change. Step 5: the new request is PENDING and appears in the approval queue.
MT-25	1. Leave the organization default as Require an approver. As admin@a.test, edit Projector and set Checkout approval to Always approve automatically; save. 2. As member@a.test, request the AVAILABLE projector unit. 3. Set the organization default to Approve automatically and set Camera	Step 2: the projector request is APPROVED immediately; the asset page shows its approval setting. Step 3: the camera request is PENDING despite the organization default - the per-asset setting wins. Step 4: the asset follows the organization default again.

	to Always require an approver; request a camera unit as member@a.test. 4. Set Camera back to Use the organization default.	
MT-26	1. Sign in as admin@a.test, open Groups and create a group named "Field Team". 2. Try to create another group named "field team". 3. Open Field Team, add member@a.test with Add to group, then rename the group to "Field Crew". 4. Remove member@a.test, then delete the group. 5. Open Audit log.	Step 1: the group appears in the list with no members. Step 2: refused - group names are unique within an organization regardless of letter case. Step 3: the member is listed and the new name shows everywhere the group appears. Step 4: the member is removed and the group disappears from the list. Step 5: one GROUP_ event for each change, naming the admin.
MT-27	1. As admin@a.test, create a group "Camera Club" with approver@a.test as its only member. 2. Edit Camera and, under Who can request this, restrict it to Camera Club. 3. Sign in as member@a.test and open the Catalog, then Camera. 4. Sign in as approver@a.test and request an AVAILABLE camera unit. 5. As admin@a.test, add member@a.test to Camera Club; as member@a.test, reload Camera.	Step 3: Camera is still listed but shows a Restricted badge; its page says "Only members of Camera Club can request this" and every Request button is disabled. Step 4: the request is accepted. Step 5: the member can now request a camera unit.
MT-28	1. As member@a.test, open the seeded APPROVED laptop request (or approve a new camera request as approver@a.test). 2. Click "I've picked it up". 3. As approver@a.test, open the same request in a tab that was opened before step 2 and click Record handoff. 4. Invite a second member (see MT-37) and, signed in as them, open the first member's request URL.	Step 2: the request becomes CHECKED_OUT, the unit OUT, and a due date is set; the history shows the pickup was recorded by the borrower. Step 3: refused with a message that the item is already checked out - the pickup is recorded once. Step 4: the second member cannot see the request and has no pickup action.
MT-29	1. As member@a.test, open the	Step 1: the request is

	CHECKED_OUT projector request and click Return this item, adding a condition note. 2. As approver@a.test, open Approvals with the Return pending filter, open the request and click Confirm return. 3. Re-seed. Repeat step 1, then as approver@a.test click Reject return instead. 4. As approver@a.test, use Record return on the CHECKED_OUT laptop (a walk-in return).	RETURN_PENDING and the unit stays OUT; the member has no Confirm button. Step 2: the request is RETURNED and the unit AVAILABLE; the page shows the condition received. Step 3: the request goes back to CHECKED_OUT and the unit stays OUT. Step 4: the request is RETURNED in one step.
MT-30	1. As approver@a.test, request an AVAILABLE camera unit. 2. As admin@a.test, approve it. 3. As approver@a.test, click "I've picked it up", then Return this item. 4. Still as approver@a.test, open the request. 5. As admin@a.test, open the request.	Step 4: the request is RETURN_PENDING and the approver is offered neither Confirm return nor Reject return on their own loan. Step 5: the admin can confirm the return.
MT-31	1. Sign in as admin@a.test and open Settings. Note the Pickup window (Hours to collect an approved item). 2. Click Expire uncollected approvals now. 3. Click it again. 4. Open Laptop, then Approvals with the Expired filter. 5. Enter 721 in the hours field, then 24, and save.	Step 1: the window is 48 hours by default. Step 2: "1 approval expired and the units released." - the seeded laptop approval (needed from 25 September) is past its window. Step 3: nothing more expires. Step 4: the laptop's HELD unit is AVAILABLE and the request is EXPIRED, with a system audit entry. Step 5: 721 cannot be saved (whole hours, up to 720); 24 saves with a confirmation.
MT-32	1. As member@a.test, open Catalog and type "cam" into Search the catalog without pressing Enter. 2. Change the text to a word that matches nothing. 3. As admin@a.test, create an asset "Tripod" with no units and retire it.	Step 1: "Searching..." appears briefly, then Camera is listed; results follow the typing without a submit. Step 2: an empty result, not an error. Step 4: no results - retired assets are left out of search.

	4. As member@a.test, search for "tripod".	
MT-33	1. As member@a.test, open Laptop (it has no AVAILABLE unit). 2. Open Camera (it has AVAILABLE units). 3. As admin@a.test, restrict Projector to a group member@a.test is not in; reload Laptop as member@a.test.	Step 1: a "Similar items available now" section lists up to five available assets (not the laptop itself), each with a reason (Foundry on) or "Available now" (Foundry off) and a View link. Step 2: no section is shown. Step 3: Projector is no longer suggested.
MT-34	1. As admin@a.test, create a group and copy its URL; copy the Laptop asset URL too. 2. Sign in as admin@b.test and open both URLs. 3. As admin@b.test, open Groups and search the catalog for "laptop".	Step 2: both show not found - no Org A data and no Org A similar items. Step 3: only Org B's groups and assets are listed.
MT-35	1. As admin@a.test, open Users and invite a new member with a name, email and initial password. 2. Sign in as the new member. 3. As admin@a.test, click Reset password for that member and note the temporary password. 4. Sign in as the member with the temporary password.	Step 1: the member appears in the list with the MEMBER role. Step 2: they are asked to set a new password before using the app. Step 4: the temporary password works once, and they must set a new password again.

3. Automated Testing Report

Where the tests live

Backend tests are in code/backend/tests, split into tests/unit (services, middleware, policies, utilities) and tests/integration (routes, plus a security folder). Frontend tests are in code/frontend/tests/unit, covering components, hooks, services and utilities. Frameworks: Vitest throughout; Supertest for API routes without starting an HTTP server; React Testing Library for components; mongodb-memory-server to run a real MongoDB replica set in memory for the duration of a run. Each backend test file gets its own database inside that replica set, with the production indexes applied by the same migrations that run in deployment, and the data is

wiped between tests. Playwright end-to-end tests remain a nice-to-have and were not written in Iteration 1.

How it runs

Every push and pull request runs, for both backend and frontend: lint, a formatting check, the test suite with coverage, and npm audit; plus a Docker image build for each. Coverage is gated - a run fails if line coverage falls below 80%, the target set in the SPPP quality assurance plan (NFR-10/11). Three test files are worth calling out because they are what makes the security design checkable rather than merely stated:

Test file	Cases	What it proves
<code>tests/integration/security/routePermissions.test.js</code>	134	Walks the built route table and checks every registered route against the permission matrix, including that each returns 401 without a session. A newly added route cannot be left unprotected by accident (SR-1).
<code>tests/integration/security/tenantIsolation.test.js</code>	27	Seeds two organizations and confirms a token from one gets 404 - not 403 - for the other's ids, at both the repository and the route layer (SR-2). All 11 endpoint rows are active; six that were still flagged unimplemented after their endpoints had shipped were enabled during Iteration 1, and all six passed.
<code>tests/unit/services/checkout.t.transitions.test.js</code>	72	Every legal and illegal transition of the checkout state machine, including what each does to the physical unit.

Iteration 2 changes and key test files

The layout and frameworks are unchanged. The backend suite grew from 32 to 45 test files (19 unit, 26 integration) and the frontend from 28 to 36. Tests never inherit a live Foundry key: the Vitest configuration forces `FOUNDRY_ENABLED=false` and an empty key, so a test that forgot to mock the client cannot make a real, billed call, and the AI paths are tested against a recorded response (`tests/fixtures/foundry/asset-search-response.json`). The backend Docker image gained a dev stage so the backend suite can also run inside the container.

The three files above at Iteration 2, and the largest new test files:

Test file	Cases	What it proves
tests/integration/security/routePermissions.test.js	229	134 cases in Iteration 1. The new group, settings, maintenance, search, alternatives and custody routes were picked up by the route-table walk with no change to the test, so none of them can ship unprotected (SR-1).
tests/integration/security/tenantIsolation.test.js	27	Unchanged in size. Cross-tenant 404s for the new endpoints are checked in each feature's own tests (maintenance, groups, alternatives) (SR-2).
tests/unit/services/checkout.transitions.test.js	108	72 cases in Iteration 1. The table-driven cases grew with the state machine, which gained OVERDUE, RETURN_PENDING and EXPIRED.
tests/integration/routes/custodyConfirmation.test.js	51	Pickup recorded by either side and only once, returns confirmed or rejected by someone other than the borrower (with the sole-confirmer fallback), and expiry of uncollected approvals (SCRUM-205).
tests/integration/routes/groups.test.js	41	Group lifecycle, case-insensitive unique names, membership changes and their audit events (SCRUM-149).
tests/integration/routes/approvalSettings.test.js	30	Organization-wide and per-asset approval modes, including that an auto-approving asset cannot let an ineligible member skip the group check (SCRUM-148, SCRUM-150).

4. Testing Metrics

4.1 Iteration 1

4.1.1 Test results and coverage

The test and coverage figures in the table below are from the CI run on main at commit e4a0efd (21 September 2026), which passed. The SPPP quality assurance plan sets line coverage at 80% or better; both packages are comfortably above it, and CI fails the build if either drops below.

Metric	Backend	Frontend
--------	---------	----------

Test files	31 passed, 1 skipped (32)	27 passed, 1 skipped (28)
Test cases	679 passed, 6 skipped (685)	242 passed, 1 todo (243)
Test case pass rate	100% of executed cases	100% of executed cases
Line coverage (target >= 80%)	94.56%	95.97%
Statement coverage	94.46%	95.79%
Branch coverage	86.72%	89.97%
Function coverage	95.59%	95.26%
Dependency audit (SR-13)	0 vulnerabilities at high or above	0 vulnerabilities at high or above
Defects open against Iteration 1	SCRUM-125 (audit immutability, database layer) - open, blocked on Atlas console access	-

On the skipped and todo cases. The 6 skipped backend cases are tests/integration/security/auditDbRole.test.js, which verifies the insert-only database role for the audit collection (SR-8, SCRUM-125). They are skipped deliberately: they need a real authenticated MongoDB deployment, and run only when AUDIT_ROLE_TEST_URI points at one. They are not a gap in effort but an open item - until that role is applied, audit immutability rests on the application layer alone. The remaining skipped file and the single frontend todo are staged acceptance criteria for work not yet implemented, per the convention above.

4.1.2 Code structure metrics

These metrics were measured separately by the QA Lead from the source tree. LOC counts exclude comments and blank lines, and the per-file figures also exclude configuration files. Fan-out counts relative imports of the project's own modules and excludes third-party libraries.

Metric	Backend	Frontend	Combined
Average LOC / file	61.9 63 files, 3,902 LOC	63.4 55 files, 3,487 LOC	62.6 118 files, 7,389 LOC
Average LOC /	13.6	29.4	18.5

function	260 functions	120 functions	380 functions
Average fan-out / file	2.6 165 internal import edges	3.1 168 internal import edges	2.8 333 internal import edges

4.2 Iteration 2

4.2.1 Test results and coverage

The backend figures in the table below are from the CI run on main at commit 5c4bcc8 (4 October 2026), which passed every job. The frontend figures are from a run of the same commit's suite with coverage, and match the totals recorded on PR #63. Both packages stay above the 80% line-coverage gate, and every coverage measure rose from Iteration 1.

Metric	Backend	Frontend
Test files	44 passed, 1 skipped (45)	35 passed, 1 skipped (36)
Test cases	1,114 passed, 6 skipped (1,120)	394 passed, 1 todo (395)
Test case pass rate	100% of executed cases	100% of executed cases
Line coverage (target >= 80%)	95.19%	97.79%
Statement coverage	95.13%	97.78%
Branch coverage	87.81%	93.08%
Function coverage	96.76%	97.76%
Dependency audit (SR-13)	0 vulnerabilities at high or above	0 vulnerabilities at high or above
Defects fixed in Iteration 2	PR #64: approval-queue rows showed "Unknown user" / "Unknown asset" after the UI rework; fixed, with 9 regression cases in requests.test.js	-
Defects open against	SCRUM-125 (audit	-

Iteration 2	immutability, database layer) - still open, blocked on Atlas console access	
--------------------	---	--

On the skipped and todo cases. These are unchanged from Iteration 1. The 6 skipped backend cases are still `auditDbRole.test.js`: the suite's in-memory MongoDB runs without authentication, so the insert-only audit role cannot be checked there, and the role has not yet been applied to any deployment (SCRUM-125). Until it is, audit immutability (SR-8) rests on the application layer, which is tested. The frontend skip and todo are staged acceptance criteria.

4.2.2 Code structure metrics

Measured from the source tree at commit 5c4bcc8 by script (a JavaScript parser), using the definitions in 4.1.2 over every .js and .jsx file under src: LOC excludes comments and blank lines, fan-out counts relative imports of the project's own modules, and LOC per function counts top-level functions, with callbacks nested inside a function counted toward it. The script counts slightly differently from the hand measurement in 4.1.2, so the comparison in 4.3 uses the same script run on the Iteration1 tag, comparing like with like.

Metric	Backend	Frontend	Combined
Average LOC / file	80.9 69 files, 5,579 LOC	74.0 71 files, 5,254 LOC	77.4 140 files, 10,833 LOC
Average LOC / function	12.2 336 functions	26.5 171 functions	17.0 507 functions
Average fan-out / file	2.9 197 internal import edges	3.2 228 internal import edges	3.0 425 internal import edges

4.3 Iteration 1 vs Iteration 2

The two iterations side by side. Each cell shows Iteration 1 then Iteration 2, with the change underneath. Test and coverage figures come from the two CI runs (e4a0efd and 5c4bcc8); the code-structure rows marked * use the script figures for both iterations, as explained in 4.2.2.

Metric	Backend	Frontend
Test files	32 → 45 +13	28 → 36 +8

Test cases	685 → 1,120 +435	243 → 395 +152
Test case pass rate	100% → 100% no change	100% → 100% no change
Line coverage	94.56% → 95.19% +0.63 pts	95.97% → 97.79% +1.82 pts
Statement coverage	94.46% → 95.13% +0.67 pts	95.79% → 97.78% +1.99 pts
Branch coverage	86.72% → 87.81% +1.09 pts	89.97% → 93.08% +3.11 pts
Function coverage	95.59% → 96.76% +1.17 pts	95.26% → 97.76% +2.50 pts
High or critical vulnerabilities	0 → 0 no change	0 → 0 no change
Source files	63 → 69 +6	55 → 71 +16
Average LOC / file *	62.9 → 80.9 +18.0	64.5 → 74.0 +9.5
Average LOC / function *	11.9 → 12.2 +0.3	26.1 → 26.5 +0.4
Average fan-out / file *	2.6 → 2.9 +0.3	2.9 → 3.2 +0.3

Iteration 2 added 435 backend and 152 frontend test cases while every coverage measure rose, so the new code arrived with proportionally more tests than the old. Average file size grew by about a fifth, mostly because Iteration 2 features were added to existing modules (checkout.service.js in particular); function length and fan-out barely moved.

5. AI Usage Log

This table summarizes AI contributions in this document.

Section	Who	AI contribution (%)	Description	Verified by
1, 3, 4	Amber (on behalf of QA)	75%	Claude Code compiled the automated testing report and the metrics table from the CI run on main and from the test suite itself. Every figure was read from the CI log rather than estimated. The manual testing report was deliberately left unfilled.	Alex Picard
4.2 and layout	Orelmis	20%	The QA Lead measured the LOC, function-length and fan-out figures. Claude formatted them into a table, drafted the surrounding text, and reworked the document layout; it did not generate or estimate any figures.	Orelmis Toribio
Iteration 2 additions (1-4)	Orelmis Toribio	85%	Claude compiled the Iteration 2 sections from the repository history since the	Orelmis Toribio

			Iteration1 tag and the CI run on main at 5c4bcc8. It read the backend figures from the CI log, ran the frontend suite with coverage, measured the code-structure metrics with a script, and drafted the Iteration 2 manual test cases. It did not run the manual tests; their results are recorded by the QA Lead.	
--	--	--	---	--

6. References

[1] Team 3, "SafeDrop Software Project Proposal and Plan (SPPP)," CS673, Boston University, Sept. 2026.

[2] Team 3, "SafeDrop Software Design Document (SDD) v0.2," CS673, Boston University, Sept. 2026.

[3] SafeDrop CI run on main at commit e4a0efd, GitHub Actions, 21 Sept. 2026 - the source of every figure in Sections 3 and 4 except the code structure metrics.

[4] code/docs/staged-acceptance-tests.md - the convention for staging acceptance criteria as tests.

[5] code/docs/audit-db-role.md - the insert-only database role for the audit collection (SCRUM-125).

[6] SafeDrop CI run on main at commit 5c4bcc8, GitHub Actions, 4 Oct. 2026 - the source of the Iteration 2 backend figures in Sections 1, 3 and 4.

[7] code/docs/std-scrum-151-test-cases.md - the SCRUM-151 test-case summary (29 automated cases).

7. Glossary

Term	Definition
Auto-approve	An approval setting under which a request is approved as soon as it is submitted; the item is still handed over in person.
Coverage gate	A CI rule that fails the build when line coverage falls below the agreed target (80% here).
Custody confirmation	The rule that a change of custody is recorded by the side that can verify it: the borrower may record a pickup, but a return is closed by someone else.
Fan-out	The number of the project's own modules that a source file imports; a measure of how tightly a file is coupled to the rest of the codebase. Third-party libraries are not counted.
Integration test	A test that exercises several layers together, here the API against a real database.
LOC (lines of code)	Non-blank, non-comment source lines.
Microsoft Foundry	The hosted AI model service behind AI-assisted search and recommendations, called only from the backend.
mongodb-memory-server	A library that starts a real MongoDB replica set in memory for the duration of a test run.
Pickup window	How long after the start of a request an approved item may be collected before the approval expires.
Regression test	A test kept in the suite so that a behaviour which once worked cannot silently break.
Restricted equipment	An asset only members of named user groups may request.
Staged acceptance test	A bodyless test case standing in for an acceptance criterion not yet implemented; Vitest reports it as todo.
Supertest	A library for calling Express routes in tests without starting an

	HTTP server.
Unit test	A test of one module with its dependencies mocked.
User group	A named set of members within an organization, used to restrict equipment.
Vitest	The test runner used for both packages.